

Mcp-Digitalinvoice — Complete Technical Architecture & Codebase Documentation

Target Repository:

Muhammad-USman-Tariq/Mcp-Digitalinvoice

System Name:

Multi-Tenant Invoice Autofill MCP Server

Authoritative Specification:

Sections 1 through 5 Detailed Breakdown

Table of Contents

1. Project Overview

2. 1.1 Purpose & Problem Solved

3. 1.2 Technology Stack

4. 1.3 High-Level System Architecture

5. 1.4 Directory Structure Breakdown

6. Routes & API Endpoints

7. 2.1 REST Endpoints Overview

8. 2.2 Detailed Route Specifications

9. 2.3 Native MCP Protocol Interface (/mcp)

10. File-by-File Breakdown

11. 3.1 Core Application Layer

12. 3.2 API Layer

13. 3.3 MCP Server Layer

14. 3.4 Data Models & Schemas

15. 3.5 Service & Session Layer

16. 3.6 Adapter Layer

17. 3.7 Database Migrations & Environment

18. Function-by-Function Breakdown

19. 4.1 src/mcp_digitalinvoice/config.py

20. 4.2 src/mcp_digitalinvoice/database.py

21. 4.3 src/mcp_digitalinvoice/logging.py

22. 4.4 src/mcp_digitalinvoice/security.py

23. 4.5 src/mcp_digitalinvoice/main.py

24. 4.6 src/mcp_digitalinvoice/models/db.py

25. 4.7 src/mcp_digitalinvoice/models/schemas.py

26. 4.8 src/mcp_digitalinvoice/adapter/reference_data.py

27. 4.9 src/mcp_digitalinvoice/adapter/exceptions.py

28. 4.10 src/mcp_digitalinvoice/adapter/mapper.py

29. 4.11 src/mcp_digitalinvoice/adapter/client.py

30. 4.12 src/mcp_digitalinvoice/session/manager.py

31. 4.13 src/mcp_digitalinvoice/services/invoice_service.py

32. 4.14 src/mcp_digitalinvoice/api/routes.py

- 33. 4.15 `src/mcp_digitalinvoice/api/setup_routes.py`
 - 34. 4.16 `src/mcp_digitalinvoice/mcp_server/middleware.py`
 - 35. 4.17 `src/mcp_digitalinvoice/mcp_server/server.py`
 - 36. Data Flow & Logic Summaries
 - 37. 5.1 Flow 1: Tenant Onboarding & API Key Issuance (`connect_account`)
 - 38. 5.2 Flow 2: End-to-End Invoice Autofill via Native MCP (`fill_invoice`)
 - 39. 5.3 Flow 3: Session Expiry, Cookie Caching, & 401 Reactive Refresh
-

1. PROJECT OVERVIEW

1.1 Purpose & Problem Solved

Mcp-Digitalinvoice is a secure, multi-tenant middleware and Model Context Protocol (MCP) server written in Python. It bridges modern AI assistants (Claude Desktop, Claude Code, OpenAI GPT via Responses API, Gemini, Google Antigravity IDE) and enterprise REST clients with third-party digital sales tax and e-invoicing platforms (specifically FBR-compliant digital invoicing systems).

The Problem:

1. **Manual Data Entry Burden:** Finance teams, accountants, and retail operators receive thousands of scanned paper receipts, PDF orders, and supplier bills that require manual re-keying into government-regulated digital invoicing portals.
2. **Credential Leakage Risk:** Passing third-party portal passwords through LLM prompts or chat conversations is an extreme security risk.
3. **Transport Protocol Incompatibilities:** Different AI clients communicate over different transports: Claude Desktop uses native streamable HTTP or local stdio; OpenAI Responses API uses bearer-authenticated webhooks; web frontends require standard OpenAPI REST endpoints.
4. **Fragile Upstream Portal Architecture:** Upstream tax software frequently drops sessions, enforces complex tax-rate lookups based on seller provinces, requires exact casing (uppercase buyer provinces), and rejects requests without company identity headers.

The Solution:

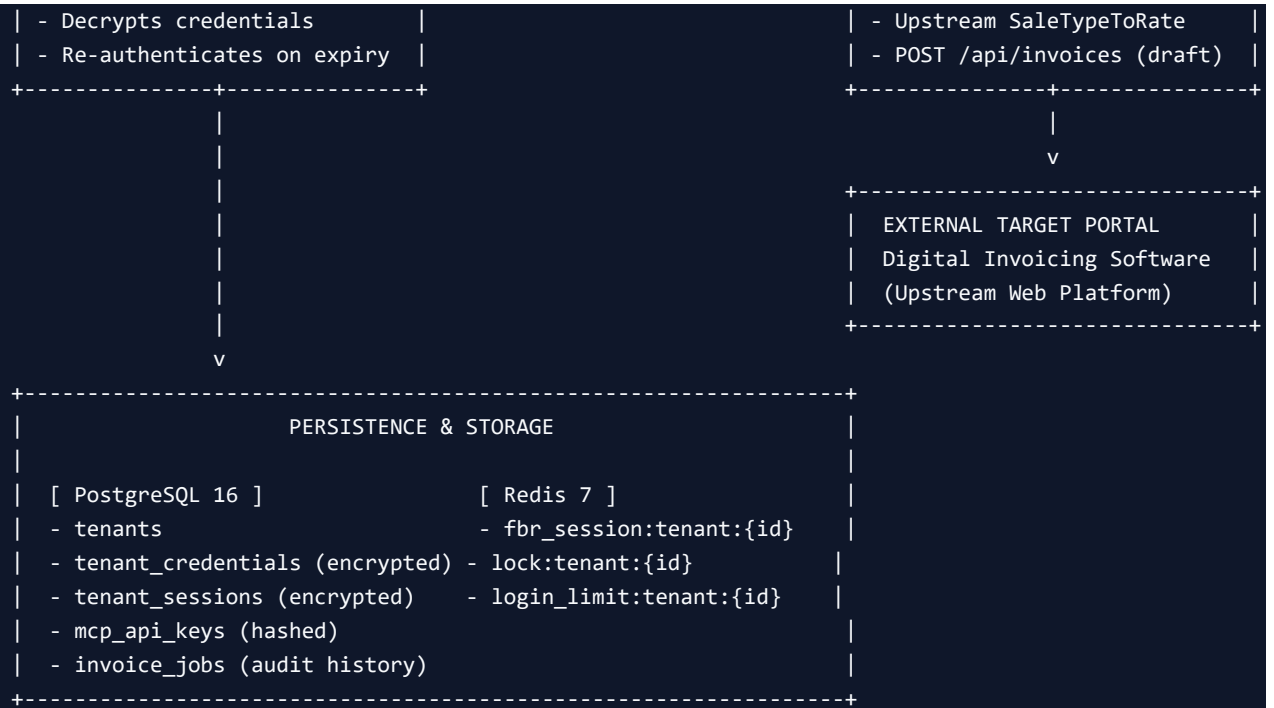
`Mcp-Digitalinvoice` unifies a **FastAPI REST server**, a **self-contained onboarding web portal (`/setup`)**, and a **stateless Streamable HTTP MCP server (`/mcp`)** into a single application process: - Stores encrypted tenant credentials at rest using Fernet symmetric encryption. - Issues unique, high-entropy MCP API keys (`mcp_*`). - Replicates browser session behavior with dynamic cookie expiration tracking and automated 401 reactive recovery. - Maps extraction models to exact upstream FBR payload schemas. - Offers per-tenant concurrency control using distributed Redis locks.

1.2 Technology Stack

Component	Technology	Version	Purpose
Language	Python	<code>>=3.12</code>	Runtime environment leveraging modern async/await and typing features.
Web Framework	FastAPI	<code>^0.110.0</code> / <code>0.141.1</code>	High-performance asynchronous web framework for REST routes and lifespan management.
ASGI Server	Uvicorn / Starlette	<code>^0.28.0</code> / <code>^1.6.0</code>	Production ASGI web server serving both HTTP REST and MCP transports.
MCP Protocol	Python MCP SDK (<code>mcp</code>)	<code>^1.29.1</code>	Model Context Protocol SDK exposing tools over Streamable HTTP and stdio.
HTTP Client	HTTPX	<code>^0.28.1</code>	Async HTTP client communicating with third-party Digital Invoicing Software.
Database ORM	SQLAlchemy (Asyncio)	<code>^2.0.52</code>	Asynchronous ORM managing relational entities and connection pools.
DB Driver	AsyncPG	<code>^0.31.0</code>	High-performance PostgreSQL driver for async I/O.
Relational DB	PostgreSQL	<code>16-alpine</code>	Persistent database storing tenants, credentials, sessions, keys, and jobs.
In-Memory Store	Redis & <code>redis-py</code>	<code>7-alpine</code> / <code>^8.1.0</code>	High-speed cache for session cookies, rate-limiting flags, and distributed locks.
Cryptography	<code>cryptography</code> (Fernet)	<code>^50.0.1</code>	AES-128-CBC symmetric encryption for sensitive tenant credentials at rest.
Validation	Pydantic & Pydantic-Settings	<code>^2.13.5</code> / <code>^2.15.0</code>	Data parsing, type safety, environment configuration, and JSON schemas.
Structured Logging	Structlog	<code>^26.1.0</code>	Zero-leakage JSON logging with automated redaction writing to <code>sys.stderr</code> .
Database Migrations	Alembic	<code>^1.19.1</code>	Automated database schema tracking, versioning, and migration runner.
Testing	Pytest, Pytest-Asyncio, Respx, Faker	<code>^9.1.1</code> / <code>^0.23.1</code>	Automated testing suite with synthetic mocking and zero-hardcoding enforcement.

1.3 High-Level System Architecture





1.4 Directory Structure Breakdown

```
Mcp-Digitalinvoice/
├── .env.example           # Environment template showing required settings without secrets
├── alembic.ini            # Alembic database migration configuration
├── docker-compose.yml     # Local PostgreSQL and Redis container orchestration
├── Dockerfile.mcp-http   # Standalone container file for independent MCP HTTP runs
├── pyproject.toml         # Project dependencies, build metadata, and pytest configuration
├── requirements.txt       # Pinned dependencies for deployment environments
├── README.md             # Quickstart, installation, and usage guide
├── migrations/           # Database schema migrations
│   ├── env.py            # Alembic async migration execution script
│   ├── script.py.mako    # Template for newly generated migration revisions
│   └── versions/
│       └── 001_initial_schema.py # Database schema creation (tenants, credentials, jobs, etc.)
├── src/
│   └── mcp_digitalinvoice/ # Main application package
│       ├── __init__.py    # Package init file exposing __version__ = "0.1.0"
│       ├── config.py      # Pydantic Settings reading environment variables
│       ├── database.py    # PostgreSQL async engine, sessionmaker, and Redis connector
│       ├── logging.py     # Structlog configuration and recursive sensitive data scrubber
│       ├── main.py        # Primary FastAPI entrypoint uniting REST and mounted MCP ASGI app
│       ├── security.py    # Fernet encryption, decryption, and SHA-256 API key hashing
│       ├── adapter/
│       │   ├── __init__.py # Adapter package initializer
│       │   ├── client.py   # Async HTTP client adapter with retry logic and cookie parsing
│       │   ├── exceptions.py # Typed exception classes (UpstreamServerError, etc.)
│       │   ├── mapper.py   # Pure transformation from camelCase extraction to internal payload
│       │   └── reference_data.py # Static lookup dictionaries for FBR provinces and transaction IDs
│       ├── api/
│       │   ├── __init__.py # API package initializer
│       │   ├── routes.py   # OpenAPI REST endpoints (/api/v1/connect_account, fill_invoice, etc)
│       │   └── setup_routes.py # Self-contained browser onboarding UI served at GET /setup
│       ├── mcp_server/
│       │   ├── __init__.py # MCP server package initializer
│       │   ├── middleware.py # ASGI middleware extracting per-request MCP authentication keys
│       │   └── server.py   # FastMCP server exposing tools (fill_invoice, validate, submit)
│       ├── models/
│       │   ├── __init__.py # Models package initializer
│       │   ├── db.py       # SQLAlchemy ORM declarative models
│       │   └── schemas.py  # Pydantic validation schemas for API and MCP contracts
│       ├── services/
│       │   ├── __init__.py # Services package initializer
│       │   └── invoice_service.py # Core InvoiceService managing onboarding, locks, and fill orchestration
│       └── session/
│           ├── __init__.py # Session package initializer
│           └── manager.py  # SessionManager handling Redis/Postgres caching and reactive refreshes
├── tests/
│   ├── conftest.py        # Pytest fixtures (async engine, sqlite, synthetic data generator)
│   ├── test_adapter.py    # Tests for HTTP client adapter, logins, and tax lookups
│   ├── test_api_and_mcp.py # Tests for REST endpoints, MCP middleware, and setup route
│   ├── test_edge_cases.py # Edge case tests (concurrency, network drops, malformed data)
│   ├── test_invoice_service.py # Service orchestration tests
│   ├── test_mapper.py     # Pure schema transformation unit tests
│   └── test_no_hardcoded_values.py # Repository scanner ensuring no credentials or business data leak
```

```
| test_security.py           # Unit tests for Fernet encryption and API key hashing
| test_session_manager.py   # Tests for session caching, TTL, and reactive invalidation
```

2. ROUTES & API ENDPOINTS

2.1 REST Endpoints Overview

HTTP Method	Path	File Defined	Authentication	Purpose
GET	/health	main.py	None	Basic liveness and environment probe.
GET	/setup	api/setup_routes.py	None	Interactive, self-contained onboarding and config web interface.
POST	/api/v1/connect_account	api/routes.py	None	Onboard new tenant, verify credentials, issue MCP API key.
POST	/api/v1/fill_invoice	api/routes.py	Bearer Token / X-API-Key	Autofill and save draft invoice via REST.
POST	/api/v1/validate_invoice	api/routes.py	Bearer Token / X-API-Key	Validation stub (out of scope).
POST	/api/v1/submit_invoice	api/routes.py	Bearer Token / X-API-Key	Final submission stub (out of scope).
POST / GET	/mcp	main.py / mcp_server/server.py	X-MCP-API-Key / x-api-key / Bearer	Native MCP Streamable-HTTP transport endpoint.

2.2 Detailed Route Specifications

1. GET /health

- **Location:** src/mcp_digitalinvoice/main.py:38
- **Purpose:** Liveness check probe used by Docker, Kubernetes, Coolify, or uptime monitors.
- **Request Parameters:** None.
- **Response Format (200 OK):** json { "status": "ok", "environment": "sandbox" }
- **Internal Calls:** Reads settings.digital_invoicing_env.
- **Middleware / Auth:** None.

2. GET /setup

- **Location:** `src/mcp_digitalinvoice/api/setup_routes.py:469`
- **Purpose:** Serves an interactive single-page onboarding web application matching Digital Invoicing Software branding. Users enter their portal credentials once and receive copy-ready JSON/curl configurations for Claude Desktop, Gemini/Antigravity, and OpenAI GPT.
- **Request Parameters:** None.
- **Response Format** (200 OK): `HTMLResponse` containing standalone CSS and JavaScript.
- **Internal Calls:** None. The page's JavaScript calls `POST /api/v1/connect_account` directly from the browser.
- **Middleware / Auth:** None.

3. POST /api/v1/connect_account

- **Location:** `src/mcp_digitalinvoice/api/routes.py:62`
- **Purpose:** Onboard a new tenant business. Validates provided portal credentials against the live third-party service, stores credentials encrypted, and generates an MCP API key.
- **Request Body** (application/json): `json { "name": "Acme Trading Co.", "email": "user@example.com", "password": "SecretPassword123!" }`
- **Response Format** (200 OK - `ConnectAccountResult`): `json { "tenant_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d", "mcp_api_key": "mcp_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX", "status": "connected", "message": "Account connected successfully. Store your MCP API key securely." }`
- **Error Response** (400 Bad Request): Returned if upstream credentials fail authentication.
- **Internal Calls:**
 - `InvoiceService.connect_account(body)`
 - `DigitalInvoicingAdapter.login(email, password)`
 - `security.encrypt_secret(...)`
 - `security.generate_mcp_api_key()` & `security.hash_api_key(...)`
 - SQLAlchemy commit saving `Tenant`, `TenantCredential`, and `MCPAPIKey`.
- **Middleware / Auth:** None (open onboarding endpoint).

4. POST /api/v1/fill_invoice

- **Location:** `src/mcp_digitalinvoice/api/routes.py:82`
- **Purpose:** REST endpoint allowing programmatic callers to trigger invoice autofill using standard HTTP POST requests.
- **Authentication:** `Authorization: Bearer <mcp_api_key>` OR `X-API-Key: <mcp_api_key>`.
- **Request Body** (application/json - `FillInvoiceInput`): `json { "buyer": { "businessName": "Lahore Fabrics Ltd", "ntnCnic": "1122334-5", "province": "Punjab", "address": "45 Ferozepur Road, Lahore", "registrationType": "Registered" }, "items": [{ "hsCode": "5407.5200", "description": "Woven polyester fabric roll", "quantity": 12, "saleType": "Goods at standard rate (default)", "uom": "Meters", "fixedValue": 350, "rate": null }], "meta": { "invoiceRefNo": "INV-2026-001", "scenarioId": "SN001", "invoiceDate": "2026-09-04" }, "sourceDocumentHash": null }`
- **Response Format** (200 OK - `FillInvoiceResult`): `json { "status": "saved", "remote_invoice_id": "c27e813e-aa69-4f89-a730-1b0bd8839fce", "missing_fields": null, "summary": "Invoice draft created successfully with remote ID c27e813e-aa69-4f89-a730-1b0bd8839fce.", "error": null }`
- **Internal Calls:**
 - `get_current_tenant_id` dependency (validates API key via SHA-256 lookup in `mcp_api_keys`).
 - `InvoiceService.fill_invoice(tenant_id, body)`.
- **Middleware / Auth:** FastAPI dependency `get_current_tenant_id`.

5. POST /api/v1/validate_invoice

- **Location:** `src/mcp_digitalinvoice/api/routes.py:98`
- **Purpose:** Validation stub against tax authority rules.
- **Authentication:** `Authorization: Bearer <mcp_api_key>`.
- **Response Format** (200 OK): `{"status": "failed", "summary": "Validate Invoice is currently out of scope...", "error": "Feature out of scope."}`.

6. POST /api/v1/submit_invoice

- **Location:** `src/mcp_digitalinvoice/api/routes.py:114`
- **Purpose:** Final submission stub.
- **Authentication:** `Authorization: Bearer <mcp_api_key>`.
- **Response Format** (200 OK): `{"status": "failed", "summary": "Submit Invoice is currently out of scope.", "error": "Feature out of scope."}`.

2.3 Native MCP Protocol Interface (/mcp)

- **Location:** `src/mcp_digitalinvoice/main.py:43` & `src/mcp_digitalinvoice/mcp_server/server.py`
- **Transport:** Streamable HTTP (spec-compliant JSON-RPC over HTTP with SSE streaming support).
- **Mounted Path:** Handled at `/mcp` via `app.mount("/", MCPAuthHeaderMiddleware(mcp_asgi_app))`.
- **Authentication:** `MCPAuthHeaderMiddleware` checks three headers in priority order:
 - `X-MCP-API-Key: <key>`
 - `x-api-key: <key>`
 - `Authorization: Bearer <key>`
- **Exposed MCP Tools:**

Tool Name	Parameters	Return Schema	Purpose
<code>fill_invoice</code>	<code>buyer: Dict</code> , <code>items: List[Dict]</code> , <code>meta: Optional[Dict]</code> , <code>source_document_hash: Optional[str]</code>	<code>Dict[str, Any]</code>	Primary tool for LLMs to extract receipt details, auto-lookup tax rates, and save an invoice draft.
<code>validate_invoice</code>	<code>invoice_id: str</code>	<code>Dict[str, Any]</code>	Stub tool returning clean out-of-scope status.
<code>submit_invoice</code>	<code>invoice_id: str</code>	<code>Dict[str, Any]</code>	Stub tool returning clean out-of-scope status.

3. FILE-BY-FILE BREAKDOWN

3.1 Core Application Layer

src/mcp_digitalinvoice/config.py

- **One-line Purpose:** Loads and validates environment configurations into a singleton typed Pydantic Settings object.
- **Imports & Dependencies:** `pydantic_settings.BaseSettings`, `SettingsConfigDict`.
- **Exports & Consumers:** Exports `settings` (instance of `Settings`). Consumed by `database.py`, `logging.py`, `security.py`, `adapter/client.py`, `session/manager.py`, and `main.py`.

src/mcp_digitalinvoice/database.py

- **One-line Purpose:** Initializes async PostgreSQL connection pooling and provides Redis async client access.
- **Imports & Dependencies:** `sqlalchemy.ext.asyncio`, `redis.asyncio.Redis`, `config.settings`.
- **Exports & Consumers:** Exports `engine`, `AsyncSessionLocal`, `get_db_session`, `get_redis_client`. Consumed by `api/routes.py`, `mcp_server/server.py`, and test fixtures.

src/mcp_digitalinvoice/logging.py

- **One-line Purpose:** Configures zero-leakage structured JSON logging to `sys.stderr` with automatic redaction of sensitive credentials.
- **Imports & Dependencies:** `sys`, `logging`, `structlog`.
- **Exports & Consumers:** Exports `logger`, `configure_logging`, `redact_sensitive_data`. Consumed throughout the entire codebase for observability.

src/mcp_digitalinvoice/security.py

- **One-line Purpose:** Provides cryptographic functions for encrypting/decrypting secrets and generating/hashing MCP API keys.
- **Imports & Dependencies:** `hashlib`, `secrets`, `cryptography.fernet.Fernet`, `config.settings`.
- **Exports & Consumers:** Exports `encrypt_secret`, `decrypt_secret`, `generate_mcp_api_key`, `hash_api_key`. Consumed by `services/invoice_service.py`, `session/manager.py`, `api/routes.py`, and `mcp_server/server.py`.

src/mcp_digitalinvoice/main.py

- **One-line Purpose:** Application entrypoint unifying REST routes, setup onboarding UI, and Streamable HTTP MCP transport into a single FastAPI instance.
 - **Imports & Dependencies:** `contextlib.asynccontextmanager`, `fastapi.FastAPI`, `config.settings`, `logging`, `api.routes.router`, `api.setup_routes.setup_router`, `mcp_server.server.mcp`, `mcp_server.middleware.MCPAuthHeaderMiddleware`, `mcp.server.transport_security.TransportSecuritySettings`.
 - **Exports & Consumers:** Exports `app`, `mcp_asgi_app`, `lifespan`. Consumed by Uvicorn in Dockerfile and ASGI test runners.
-

3.2 API Layer

src/mcp_digitalinvoice/api/routes.py

- **One-line Purpose:** Exposes OpenAPI-compliant REST endpoints under `/api/v1` for tenant onboarding, invoice filling, and validation.
- **Imports & Dependencies:** `fastapi`, `sqlalchemy`, `redis.asyncio`, `database`, `security`, `models.db`, `models.schemas`, `services.invoice_service`.
- **Exports & Consumers:** Exports `router`, `get_current_tenant_id`. Consumed by `main.py`.

src/mcp_digitalinvoice/api/setup_routes.py

- **One-line Purpose:** Serves an inline, zero-dependency HTML/CSS/JS onboarding portal at `GET /setup` matching Digital Invoicing Software design aesthetic.
- **Imports & Dependencies:** `fastapi.APIRouter`, `fastapi.responses.HTMLResponse`.
- **Exports & Consumers:** Exports `setup_router`, `SETUP_HTML_TEMPLATE`. Consumed by `main.py`.

3.3 MCP Server Layer

src/mcp_digitalinvoice/mcp_server/middleware.py

- **One-line Purpose:** Captures per-request API key headers (`X-MCP-API-Key`, `x-api-key`, `Authorization: Bearer`) and sets an async ContextVar.
- **Imports & Dependencies:** `contextvars`, `typing`.
- **Exports & Consumers:** Exports `mcp_api_key_ctx`, `MCPAuthHeaderMiddleware`. Consumed by `main.py` and `mcp_server/server.py`.

src/mcp_digitalinvoice/mcp_server/server.py

- **One-line Purpose:** Defines native FastMCP tools (`fill_invoice`, `validate_invoice`, `submit_invoice`) and provides stdio fallback execution.
- **Imports & Dependencies:** `os`, `asyncio`, `mcp.server.fastmcp.FastMCP`, `sqlalchemy`, `database`, `security`, `models.db`, `models.schemas`, `services.invoice_service`, `logging`, `mcp_server.middleware`.
- **Exports & Consumers:** Exports `mcp`, `fill_invoice`, `validate_invoice`, `submit_invoice`, `_resolve_tenant_id`, `main`. Consumed by `main.py` and CLI runners.

3.4 Data Models & Schemas

src/mcp_digitalinvoice/models/db.py

- **One-line Purpose:** Declares SQLAlchemy ORM models representing Postgres relational tables (`tenants`, `tenant_credentials`, `tenant_sessions`, `mcp_api_keys`, `invoice_jobs`).
- **Imports & Dependencies:** `uuid`, `datetime`, `sqlalchemy.orm`, `sqlalchemy`.
- **Exports & Consumers:** Exports `Base`, `Tenant`, `TenantCredential`, `TenantSession`, `MCPAPIKey`, `InvoiceJob`, `utc_now`. Consumed by `services/invoice_service.py`, `session/manager.py`, `api/routes.py`, and `mcp_server/server.py`.

src/mcp_digitalinvoice/models/schemas.py

- **One-line Purpose:** Declares Pydantic schemas validating client inputs and formatting API/MCP tool responses.

- **Imports & Dependencies:** `pydantic.BaseModel`, `Field`, `ConfigDict`.
- **Exports & Consumers:** Exports `BuyerInfo`, `InvoiceItem`, `InvoiceMeta`, `FillInvoiceInput`, `ConnectAccountInput`, `ConnectAccountResult`, `FillInvoiceResult`, `GenericToolResult`. Consumed across all layers.

3.5 Service & Session Layer

`src/mcp_digitalinvoice/services/invoice_service.py`

- **One-line Purpose:** Orchestrates invoice autofill workflows, credential validation, distributed locking, tax rate lookups, and draft persistence.
- **Imports & Dependencies:** `hashlib`, `json`, `uuid`, `datetime`, `redis.asyncio`, `sqlalchemy`, `config`, `security`, `models.db`, `models.schemas`, `session.manager.SessionManager`, `adapter.client.DigitalInvoicingAdapter`, `adapter.mapper`, `adapter.exceptions`.
- **Exports & Consumers:** Exports `InvoiceService`. Consumed by `api/routes.py` and `mcp_server/server.py`.

`src/mcp_digitalinvoice/session/manager.py`

- **One-line Purpose:** Implements two-tier session caching (Redis + PostgreSQL), dynamic cookie TTL tracking, rate-limiting, and reactive 401 re-login.
- **Imports & Dependencies:** `datetime`, `json`, `uuid`, `redis.asyncio`, `sqlalchemy`, `config`, `security`, `models.db`, `adapter.client`, `adapter.exceptions`.
- **Exports & Consumers:** Exports `SessionManager`. Consumed by `services/invoice_service.py`.

3.6 Adapter Layer

`src/mcp_digitalinvoice/adapter/client.py`

- **One-line Purpose:** Asynchronous HTTP client communicating directly with Digital Invoicing Software endpoints (`/api/auth/login`, `/api/invoices`, `/api/fbr/pdi/v2/SaleTypeToRate`).
- **Imports & Dependencies:** `re`, `datetime`, `httpx`, `config`, `adapter.reference_data`, `adapter.exceptions`, `logging`.
- **Exports & Consumers:** Exports `DigitalInvoicingAdapter`. Consumed by `session/manager.py` and `services/invoice_service.py`.

`src/mcp_digitalinvoice/adapter/mapper.py`

- **One-line Purpose:** Pure schema translation mapping external camelCase invoice data and live seller profile to upstream snake_case structure.
- **Imports & Dependencies:** `datetime`, `typing`, `models.schemas`, `adapter.exceptions.MissingSellerProfileError`.
- **Exports & Consumers:** Exports `map_fbr_schema_to_internal_payload`. Consumed by `services/invoice_service.py`.

`src/mcp_digitalinvoice/adapter/reference_data.py`

- **One-line Purpose:** Static lookup tables and key normalizers for FBR government province codes and transaction type IDs.
- **Imports & Dependencies:** `typing.Optional`.

- **Exports & Consumers:** Exports `lookup_province_code`, `lookup_trans_type_id`, `normalize_lookup_key`. Consumed by `adapter/client.py`.

`src/mcp_digitalinvoice/adapter/exceptions.py`

- **One-line Purpose:** Typed custom exceptions representing domain-specific failure modes across the upstream integration boundary.
- **Imports & Dependencies:** Standard library `Exception`.
- **Exports & Consumers:** Exports `AdapterError`, `UpstreamContractError`, `AuthenticationError`, `UpstreamServerError`, `RateLimitError`, `UnknownSaleTypeError`, `UnknownProvinceError`, `AmbiguousRateError`, `MissingSellerProfileError`, `ConnectionBrokenError`. Consumed across `adapter`, `session`, and `services`.

3.7 Database Migrations & Environment

`migrations/env.py` & `migrations/versions/001_initial_schema.py`

- **One-line Purpose:** Alembic migration scripts defining and applying asynchronous DDL schemas for PostgreSQL tables and indexes.
- **Imports & Dependencies:** `alembic`, `sqlalchemy`.

`docker-compose.yml`

- **One-line Purpose:** Provisions local containerized PostgreSQL 16 (port 5432) and Redis 7 (port 6379) infrastructure with persistent volumes.

`Dockerfile.mcp-http`

- **One-line Purpose:** Container image definition packaging the application for standalone Streamable-HTTP execution on port 8001.

4. FUNCTION-BY-FUNCTION BREAKDOWN

4.1 `src/mcp_digitalinvoice/config.py`

- **Settings class:** Extends `pydantic_settings.BaseSettings`. Automatically loads variables from `.env` and system environment.
- *No standalone functions; encapsulates configuration attributes.*

4.2 `src/mcp_digitalinvoice/database.py`

`get_db_session() -> AsyncGenerator[AsyncSession, None]`

- **Purpose:** FastAPI dependency yielding an asynchronous SQLAlchemy database session from `AsyncSessionLocal` within an active transaction scope.
- **Parameters:** None.
- **Return Value:** `AsyncGenerator[AsyncSession, None]` yielding a connected `AsyncSession`.

- **Side Effects:** Opens database connection pool slot; automatically commits or closes on exit.
- **Callers:** Injected via `Depends(get_db_session)` into `api/routes.py` endpoints.

`get_redis_client()` -> `Optional[Redis]`

- **Purpose:** Creates and returns a connected async Redis client instance, or safely returns `None` if Redis is unreachable.
- **Parameters:** None.
- **Return Value:** `Optional[Redis]` — connected client or `None`.
- **Side Effects:** Attempts network socket connection to Redis URL.
- **Callers:** Injected into `api/routes.py` endpoints and initialized in `mcp_server/server.py`.

4.3 `src/mcp_digitalinvoice/logging.py`

`redact_sensitive_data(logger, method_name, event_dict)` -> `dict`

- **Purpose:** Structlog processor that recursively traverses log records and masks any key matching `REDACTED_KEYS` with `[REDACTED]`.
- **Parameters:**
- `logger`: The structlog logger instance.
- `method_name` (`str`): Logging method name (`info`, `error`, etc.).
- `event_dict` (`dict`): The dictionary of log parameters.
- **Return Value:** `dict` with sensitive values masked.
- **Side Effects:** In-place scrubbing of log dictionaries.
- **Callers:** Invoked automatically by Structlog pipeline on every log statement.

`configure_logging(log_level: str = "INFO")` -> `None`

- **Purpose:** Configures Structlog with JSON rendering, ISO timestamps, redaction filtering, and binds output specifically to `sys.stderr` so stdio MCP transport on `stdout` is never corrupted.
- **Parameters:** `log_level` (`str`): Log severity threshold (default: `"INFO"`).
- **Return Value:** `None`.
- **Side Effects:** Modifies global Structlog configuration state.
- **Callers:** `main.py:lifespan()`, `mcp_server/server.py:main()`.

4.4 `src/mcp_digitalinvoice/security.py`

`_get_fernet()` -> `Fernet`

- **Purpose:** Internal factory initializing a `cryptography.fernet.Fernet` symmetric cipher from `settings.encryption_key`.
- **Parameters:** None.
- **Return Value:** `Fernet` cipher object.
- **Side Effects:** None.
- **Callers:** `encrypt_secret()`, `decrypt_secret()`.

`encrypt_secret(plaintext: str) -> str`

- **Purpose:** Encrypts plaintext sensitive credentials (passwords, emails, session cookies) before storing in PostgreSQL.
- **Parameters:** `plaintext` (`str`): Plaintext secret.
- **Return Value:** `str` : Base64-encoded encrypted Fernet token.
- **Side Effects:** None.
- **Callers:** `InvoiceService.connect_account()` , `SessionManager.get_valid_cookie_and_profile()` .

`decrypt_secret(ciphertext: str) -> str`

- **Purpose:** Decrypts stored Fernet tokens back into plaintext for upstream authentication.
- **Parameters:** `ciphertext` (`str`): Encrypted token string.
- **Return Value:** `str` : Decrypted plaintext.
- **Side Effects:** None.
- **Callers:** `SessionManager.get_valid_cookie_and_profile()` .

`generate_mcp_api_key() -> str`

- **Purpose:** Generates a high-entropy cryptographically secure raw API key for tenant MCP client authentication.
- **Parameters:** None.
- **Return Value:** `str` : String formatted as `mcp_<32-byte-token>` .
- **Side Effects:** Consumes system entropy via `secrets.token_urlsafe` .
- **Callers:** `InvoiceService.connect_account()` .

`hash_api_key(raw_key: str) -> str`

- **Purpose:** Computes SHA-256 hex digest of raw API key for secure storage and constant-time database matching.
- **Parameters:** `raw_key` (`str`): The raw `mcp_*` API key string.
- **Return Value:** `str` : 64-character hexadecimal SHA-256 hash.
- **Side Effects:** None.
- **Callers:** `InvoiceService.connect_account()` , `api/routes.py:get_current_tenant_id()` , `mcp_server/server.py:_resolve_tenant_id()` .

4.5 `src/mcp_digitalinvoice/main.py`

`lifespan(app: FastAPI)`

- **Purpose:** Asynchronous context manager managing application startup and shutdown lifecycle. Initializes logging and starts the MCP session manager lifecycle (`async with mcp.session_manager.run():`).
- **Parameters:** `app` (`FastAPI`): The FastAPI application instance.
- **Return Value:** Async generator / context manager.
- **Side Effects:** Starts background tasks for MCP sessions; logs startup and shutdown events.
- **Callers:** Invoked automatically by Uvicorn/ASGI server.

`health_check()`

- **Purpose:** Handles `GET /health` endpoint for monitoring infrastructure health.
- **Parameters:** None.
- **Return Value:** `dict` : `{"status": "ok", "environment": ...}` .

- **Side Effects:** None.
 - **Callers:** HTTP GET requests from load balancers or uptime monitors.
-

4.6 `src/mcp_digitalinvoice/models/db.py`

`utc_now() -> datetime`

- **Purpose:** Returns current UTC timestamp ensuring consistent timezone-aware timestamps across ORM models.
 - **Parameters:** None.
 - **Return Value:** `datetime`: Current UTC time with `timezone.utc`.
 - **Callers:** Default factories in `Tenant`, `TenantCredential`, `TenantSession`, `MCPAPIKey`, `InvoiceJob`.
-

4.7 `src/mcp_digitalinvoice/models/schemas.py`

- **Schemas:** `BuyerInfo`, `InvoiceItem`, `InvoiceMeta`, `FillInvoiceInput`, `ConnectAccountInput`, `ConnectAccountResult`, `FillInvoiceResult`, `GenericToolResult`.
 - Encapsulate data validation and serialization logic using Pydantic V2.
-

4.8 `src/mcp_digitalinvoice/adapters/reference_data.py`

`normalize_lookup_key(raw: str) -> str`

- **Purpose:** Cleans and standardizes province and sale type strings (strips whitespace, collapses multiple internal spaces, uppercases) to ensure reliable matching against government reference tables.
- **Parameters:** `raw` (`str`): Unsanitized input string.
- **Return Value:** `str`: Normalized uppercase string.
- **Callers:** `lookup_province_code()`, `lookup_trans_type_id()`.

`lookup_province_code(province_name: str) -> Optional[int]`

- **Purpose:** Resolves normalized province name to integer supplier/province ID expected by upstream endpoints (e.g., `"PUNJAB"` -> `7`).
- **Parameters:** `province_name` (`str`): Name of province.
- **Return Value:** `Optional[int]`: Numeric code or `None` if unmapped.
- **Callers:** `DigitalInvoicingAdapter.fetch_sales_tax_rate()`.

`lookup_trans_type_id(sale_type_name: str) -> Optional[int]`

- **Purpose:** Resolves normalized sale type string to integer transaction type ID (e.g., `"GOODS AT STANDARD RATE (DEFAULT)"` -> `75`).
 - **Parameters:** `sale_type_name` (`str`): Name of transaction/sale type.
 - **Return Value:** `Optional[int]`: Numeric code or `None` if unmapped.
 - **Callers:** `DigitalInvoicingAdapter.fetch_sales_tax_rate()`.
-

4.9 `src/mcp_digitalinvoice/adapter/exceptions.py`

- Declares domain exception classes: `AdapterError`, `UpstreamContractError`, `AuthenticationError`, `UpstreamServerError`, `RatelimitError`, `UnknownSaleTypeError`, `UnknownProvinceError`, `AmbiguousRateError`, `MissingSellerProfileError`, `ConnectionBrokenError`.

4.10 `src/mcp_digitalinvoice/adapter/mapper.py`

`_parse_rate_percent(raw_rate: Any) -> float`

- **Purpose:** Safely converts string rates (`"18%"`, `"18"`), integers, floats, or None into a clean float rate (e.g. `18.0`).
- **Parameters:** `raw_rate` (`Any`): Raw rate representation.
- **Return Value:** `float` : Clean percentage float.
- **Callers:** `map_fbr_schema_to_internal_payload()`.

`_parse_number(raw: Any) -> float`

- **Purpose:** Safely converts numeric quantities or fixed unit values to float, defaulting to `0.0` on malformed inputs.
- **Parameters:** `raw` (`Any`): Raw number representation.
- **Return Value:** `float`.
- **Callers:** `map_fbr_schema_to_internal_payload()`.

`map_fbr_schema_to_internal_payload(input_data: FillInvoiceInput, seller_profile: Dict[str, Any]) -> Dict[str, Any]`

- **Purpose:** Transforms camelCase LLM extraction payload into exact snake_case JSON required by upstream `POST /api/invoices`. Injects seller profile information, calculates item line values, applies sales tax formulas, and populates `status="draft"` and `total_amount`.
- **Parameters:**
- `input_data` (`FillInvoiceInput`): Validated invoice input from client/LLM.
- `seller_profile` (`Dict[str, Any]`): Active seller company metadata from session.
- **Return Value:** `Dict[str, Any]` : Structured dictionary `{"invoice": invoice_data, "details": details_data}`.
- **Side Effects:** Raises `MissingSellerProfileError` if seller fields or company/user IDs are absent.
- **Callers:** `InvoiceService.fill_invoice()`.

4.11 `src/mcp_digitalinvoice/adapter/client.py`

`DigitalInvoicingAdapter.__init__(base_url=None, http_client=None)`

- **Purpose:** Initializes adapter instance with target base URL and optional custom HTTP client.

`DigitalInvoicingAdapter._get_headers() -> Dict[str, str]`

- **Purpose:** Returns browser-like Origin, Referer, and User-Agent headers required by upstream anti-bot/CORS filters.
- **Return Value:** `Dict[str, str]` : Headers dictionary.

- **Callers:** `_request()` .

DigitalInvoicingAdapter._request(method, path, headers=None, json_body=None) -> httpx.Response

- **Purpose:** Core async HTTP transport wrapper handling URL resolution, headers, timeouts, and connection execution via `httpx.AsyncClient` .
- **Parameters:**
- `method` (`str`): HTTP verb (`GET` , `POST`).
- `path` (`str`): Relative URL endpoint path.
- `headers` (`Optional[Dict[str, str]]`): Optional custom headers (e.g. Cookie).
- `json_body` (`Optional[Dict[str, Any]]`): Optional JSON request payload.
- **Return Value:** `httpx.Response` .
- **Side Effects:** Network I/O over HTTP/HTTPS.
- **Callers:** `login()` , `create_or_update_invoice()` , `fetch_sales_tax_rate()` .

DigitalInvoicingAdapter.login(email: str, password: str) -> Dict[str, Any]

- **Purpose:** Sends `POST /api/auth/login` to third-party portal, extracts `fbr_session` cookie and `Max-Age` , parses seller company profile, user ID, and company ID.
- **Parameters:**
- `email` (`str`): Portal username/email.
- `password` (`str`): Portal plaintext password.
- **Return Value:** `Dict[str, Any]` containing `cookie` , `expires_at` , `max_age` , and `profile` .
- **Side Effects:** Network call; raises `AuthenticationError` , `UpstreamServerError` , or `AdapterError` .
- **Callers:** `InvoiceService.connect_account()` , `SessionManager.get_valid_cookie_and_profile()` .

DigitalInvoicingAdapter.create_or_update_invoice(cookie: str, payload: Dict[str, Any]) -> Dict[str, Any]

- **Purpose:** Sends `POST /api/invoices` to create or update draft invoices on the target platform. Features automatic transport retries (up to 2 retries) for transient network drops and validates that invoice status is `"draft"` .
- **Parameters:**
- `cookie` (`str`): Valid session cookie.
- `payload` (`Dict[str, Any]`): Complete invoice payload from `mapper.py` .
- **Return Value:** `Dict[str, Any]` : Upstream response dictionary containing `id` .
- **Side Effects:** Network POST creating a remote invoice draft.
- **Callers:** `InvoiceService.fill_invoice()` .

DigitalInvoicingAdapter._format_date_for_rate_lookup(invoice_date: Optional[str]) -> str

- **Purpose:** Converts standard ISO dates (`2026-09-04`) to the target site's lookup format (`4-September-2026`).
- **Parameters:** `invoice_date` (`Optional[str]`): ISO date string.
- **Return Value:** `str` : Formatted date string.
- **Callers:** `fetch_sales_tax_rate()` .

DigitalInvoicingAdapter.fetch_sales_tax_rate(cookie: str, sale_type: str, seller_province: str, invoice_date: Optional[str] = None) -> float

- **Purpose:** Queries upstream `/api/fbr/pdi/v2/SaleTypeToRate` to resolve the tax percentage for items lacking an explicit rate.
- **Parameters:**
- `cookie` (`str`): Active session cookie.

- `sale_type` (`str`): Normalized transaction sale type.
- `seller_province` (`str`): Originating seller province name.
- `invoice_date` (`Optional[str]`): Invoice date string.
- **Return Value:** `float` : Resolved tax percentage (e.g. `18.0`).
- **Side Effects:** Network GET; raises `UnknownSaleTypeError` , `UnknownProvinceError` , `AmbiguousRateError` .
- **Callers:** `InvoiceService.fill_invoice()` .

`DigitalInvoicingAdapter.validate_invoice(cookie, invoice_id)` & `submit_invoice(cookie, invoice_id)`

- **Purpose:** Stubs raising `NotImplementedError` as validation and submission are out of scope.

4.12 `src/mcp_digitalinvoice/session/manager.py`

`SessionManager.__init__(db, redis=None, adapter=None)`

- **Purpose:** Injects database session, Redis client, and adapter instance.

`SessionManager.get_valid_cookie(tenant_id: uuid.UUID) -> str`

- **Purpose:** Convenience wrapper returning only the valid cookie string for a tenant.
- **Parameters:** `tenant_id` (`uuid.UUID`): Tenant primary key.
- **Return Value:** `str` : Cookie value.
- **Callers:** Can be used by callers needing only cookie authentication.

`SessionManager.get_valid_cookie_and_profile(tenant_id: uuid.UUID) -> Tuple[str, Dict[str, Any]]`

- **Purpose:** Retrieves valid session cookie and company profile for tenant. Checks Redis cache (L1), falls back to Postgres `tenant_sessions` (L2), and if expired or missing, executes an authenticated re-login with 30s rate-limiting, updates both caches, and returns decrypted cookie and profile.
- **Parameters:** `tenant_id` (`uuid.UUID`): Tenant identifier.
- **Return Value:** `Tuple[str, Dict[str, Any]]` : (decrypted_cookie, seller_profile) .
- **Side Effects:** Reads/writes Redis; reads/writes PostgreSQL; may trigger upstream `adapter.login()` .
- **Callers:** `InvoiceService.fill_invoice()` .

`SessionManager.invalidate_cookie(tenant_id: uuid.UUID) -> None`

- **Purpose:** Flushes cached session from Redis and deletes `tenant_sessions` row in PostgreSQL upon upstream 401 Unauthorized, forcing a fresh login on the next attempt.
- **Parameters:** `tenant_id` (`uuid.UUID`): Tenant identifier.
- **Return Value:** `None` .
- **Side Effects:** Deletes Redis key; deletes PostgreSQL record and commits transaction.
- **Callers:** `InvoiceService.fill_invoice()` during reactive 401 retry loop.

4.13 `src/mcp_digitalinvoice/services/invoice_service.py`

`InvoiceService.__init__(db, redis=None, adapter=None)`

- **Purpose:** Injects DB session, Redis client, adapter, and instantiates associated `SessionManager` .

InvoiceService.connect_account(input_data: ConnectAccountInput) -> ConnectAccountResult

- **Purpose:** Onboards a tenant by performing an initial test login with Digital Invoicing Software, creating the `Tenant` row, encrypting credentials in `TenantCredential`, and generating/hashing an `MCPAPIKey`.
- **Parameters:** `input_data` (`ConnectAccountInput`): Credentials and company name.
- **Return Value:** `ConnectAccountResult`: Issued `tenant_id` and raw `mcp_api_key`.
- **Side Effects:** Upstream login request; inserts records into `tenants`, `tenant_credentials`, `mcp_api_keys`.
- **Callers:** `api/routes.py:connect_account_endpoint()`.

InvoiceService._validate_fill_input(input_data: FillInvoiceInput) -> List[str]

- **Purpose:** Validates presence of all mandatory buyer attributes (`businessName`, `province`, `registrationType`, and `ntnCnic` if registered) and line item fields (`hsCode`, `quantity`, `fixedValue`, `saleType`).
- **Parameters:** `input_data` (`FillInvoiceInput`): Submitted invoice input.
- **Return Value:** `List[str]`: List of missing field names (empty if valid).
- **Callers:** `fill_invoice()`.

InvoiceService._check_sanity_bounds(input_data: FillInvoiceInput) -> Optional[str]

- **Purpose:** Ensures numeric sanity of item quantities and unit fixed values (must be positive numbers).
- **Parameters:** `input_data` (`FillInvoiceInput`): Submitted invoice input.
- **Return Value:** `Optional[str]`: Error description if bounds fail, or `None`.
- **Callers:** `fill_invoice()`.

InvoiceService.fill_invoice(tenant_id: uuid.UUID, input_data: FillInvoiceInput) -> FillInvoiceResult

- **Purpose:** Complete orchestrator for invoice filling:
- Validates required fields and sanity bounds.
- Computes `doc_hash` (SHA-256) for audit traceability.
- Acquires per-tenant Redis distributed lock `lock:tenant:{tenant_id}`.
- Creates `InvoiceJob` in "pending" status.
- Resolves active cookie and seller profile via `SessionManager`.
- Auto-fetches missing item tax rates via `DigitalInvoicingAdapter`.
- Maps schema to internal upstream shape via `mapper.py`.
- Calls `DigitalInvoicingAdapter.create_or_update_invoice()`, handling reactive 401 cookie refresh and retry.
- Updates `InvoiceJob` status to "saved" or "failed", releases distributed lock, and returns result.
- **Parameters:**
- `tenant_id` (`uuid.UUID`): Tenant identifier.
- `input_data` (`FillInvoiceInput`): Validated invoice input payload.
- **Return Value:** `FillInvoiceResult`: Status (`"saved"`, `"needs_info"`, `"failed"`), remote invoice ID, summary.
- **Side Effects:** Writes `invoice_jobs` to DB; acquires/releases Redis lock; calls external APIs.
- **Callers:** `api/routes.py:fill_invoice_endpoint()`, `mcp_server/server.py:fill_invoice()`.

InvoiceService.validate_invoice(tenant_id, invoice_id) & submit_invoice(tenant_id, invoice_id)

- **Purpose:** Returns `GenericToolResult` with status `"failed"` noting out-of-scope feature.

4.14 `src/mcp_digitalinvoice/api/routes.py`

`get_current_tenant_id(credentials, x_api_key, db) -> uuid.UUID`

- **Purpose:** FastAPI authentication dependency resolving tenant ID by extracting API key from `Authorization: Bearer` or `X-API-Key`, computing its SHA-256 hash, and looking up active keys in `mcp_api_keys`.
- **Parameters:**
- `credentials`: Optional `HTTPAuthorizationCredentials`.
- `x_api_key`: Optional string from `X-API-Key` header.
- `db`: Injected `AsyncSession`.
- **Return Value:** `uuid.UUID`: Resolved `tenant_id`.
- **Side Effects:** Raises `HTTPException(401)` if key is missing, revoked, or invalid.
- **Callers:** `fill_invoice_endpoint`, `validate_invoice_endpoint`, `submit_invoice_endpoint`.

`connect_account_endpoint(body, db, redis) -> ConnectAccountResult`

- **Purpose:** REST handler for `POST /api/v1/connect_account`. Delegates to `InvoiceService.connect_account`.

`fill_invoice_endpoint(body, tenant_id, db, redis) -> FillInvoiceResult`

- **Purpose:** REST handler for `POST /api/v1/fill_invoice`. Delegates to `InvoiceService.fill_invoice`.

`validate_invoice_endpoint(...)` & `submit_invoice_endpoint(...)`

- **Purpose:** REST handlers for validation and submission stubs.
-

4.15 `src/mcp_digitalinvoice/api/setup_routes.py`

`setup_page() -> HTMLResponse`

- **Purpose:** Handler for `GET /setup`. Returns `SETUP_HTML_TEMPLATE` delivering the browser onboarding UI.
 - **Parameters:** None.
 - **Return Value:** `HTMLResponse`.
 - **Callers:** Direct browser requests to `/setup`.
-

4.16 `src/mcp_digitalinvoice/mcp_server/middleware.py`

`MCPAuthHeaderMiddleware.__init__(app)`

- **Purpose:** Wraps downstream Starlette/FastMCP ASGI app.

`MCPAuthHeaderMiddleware.__call__(scope, receive, send)`

- **Purpose:** Intercepts incoming ASGI HTTP requests. Inspects headers in priority order (`X-MCP-API-Key`, `x-api-key`, `Authorization: Bearer`), stores token in `mcp_api_key_ctx`, passes control downstream, and cleans up `ContextVar` in a `finally` block.
 - **Parameters:** Standard ASGI `scope`, `receive`, `send`.
 - **Side Effects:** Modifies context variable `mcp_api_key_ctx`.
 - **Callers:** Mounted in `main.py` around `mcp_asgi_app`.
-

4.17 `src/mcp_digitalinvoice/mcp_server/server.py`

`_resolve_tenant_id(db) -> Any`

- **Purpose:** Resolves tenant ID for native MCP tool executions. Reads raw key from `mcp_api_key_ctx` (HTTP transport) or falls back to `TENANT_MCP_API_KEY` environment variable (stdio transport), hashes it, and queries `mcp_api_keys`.
- **Parameters:** `db`: Active SQLAlchemy `AsyncSession`.
- **Return Value:** `uuid.UUID`: Resolved `tenant_id`.
- **Side Effects:** Raises `ValueError` if missing or invalid.
- **Callers:** `fill_invoice()`, `validate_invoice()`, `submit_invoice()`.

`fill_invoice(buyer, items, meta=None, source_document_hash=None) -> Dict[str, Any]`

- **Purpose:** Native MCP Tool annotated with `@mcp.tool()`. Opens a scoped database session and Redis client, resolves tenant ID, transforms dict parameters into Pydantic models, and executes `InvoiceService.fill_invoice()`.
- **Parameters:**
 - `buyer` (`Dict[str, Any]`): Buyer details object.
 - `items` (`List[Dict[str, Any]]`): Array of line item objects.
 - `meta` (`Optional[Dict[str, Any]]`): Invoice metadata object.
 - `source_document_hash` (`Optional[str]`): Optional idempotency/traceability hash.
- **Return Value:** `Dict[str, Any]`: Result dictionary (`status`, `remote_invoice_id`, `summary`, `error`).
- **Callers:** Invoked by LLM clients (Claude, GPT, Gemini) over MCP protocol.

`validate_invoice(invoice_id: str) -> Dict[str, Any]` & `submit_invoice(invoice_id: str) -> Dict[str, Any]`

- **Purpose:** MCP tool stubs for validation and submission.

`main()`

- **Purpose:** CLI entrypoint. Defaults to stdio transport for local tools like Claude Desktop, or runs standalone Uvicorn server if `MCP_TRANSPORT=streamable-http`.
- **Callers:** Executed via `python -m mcp_digitalinvoice.mcp_server.server`.

5. DATA FLOW / LOGIC SUMMARY

5.1 Flow 1: Tenant Onboarding & API Key Issuance (`connect_account`)

```
User (Browser /setup or curl)
|
| 1. POST /api/v1/connect_account {name, email, password}
v
[api/routes.py: connect_account_endpoint]
|
| 2. Instantiate InvoiceService(db, redis)
v
[services/invoice_service.py: connect_account]
|
| 3. Test login with credentials
v
[adapter/client.py: login] ---> POST /api/auth/login ---> [Digital Invoicing Software]
|
|<----- 200 OK + Set-Cookie: fbr_session=... <-----+
v
[services/invoice_service.py: connect_account]
| 4. Create Tenant(name=...)
| 5. Encrypt email & password -> security.encrypt_secret()
| 6. Create TenantCredential(tenant_id, encrypted_email, encrypted_password)
| 7. Generate API Key -> security.generate_mcp_api_key() => "mcp_abc123..."
| 8. Hash API Key -> security.hash_api_key("mcp_abc123...") => "e3b0c44..."
| 9. Create MCPAPIKey(tenant_id, hashed_key="e3b0c44...")
| 10. Commit database transaction (PostgreSQL)
v
User receives 200 OK: {tenant_id, mcp_api_key: "mcp_abc123...", status: "connected"}
```

5.2 Flow 2: End-to-End Invoice Autofill via Native MCP (`fill_invoice`)

```
LLM Client (Claude / GPT / Gemini)
|
| 1. POST /mcp (JSON-RPC call: tool="fill_invoice", headers: X-MCP-API-Key: "mcp_abc123...")
v
[mcp_server/middleware.py: MCPAuthHeaderMiddleware]
| 2. Extracts "mcp_abc123..." and sets mcp_api_key_ctx
v
[mcp_server/server.py: fill_invoice]
| 3. Opens AsyncSessionLocal() & get_redis_client()
| 4. Calls _resolve_tenant_id(db) -> queries mcp_api_keys where hashed_key == hash("mcp_abc123...")
| 5. Converts raw buyer, items, meta dicts into Pydantic models
v
[services/invoice_service.py: fill_invoice]
| 6. Validates required fields (_validate_fill_input) & sanity bounds
| 7. Computes SHA-256 doc_hash for traceability
| 8. Acquires Redis distributed lock: lock:tenant:{tenant_id} (timeout=30s)
| 9. Inserts InvoiceJob(status="pending", source_document_hash=doc_hash)
|
| 10. Calls SessionManager.get_valid_cookie_and_profile(tenant_id)
v
[session/manager.py: get_valid_cookie_and_profile]
| Checks Redis cache (fbr_session:tenant:{tenant_id}) -> Cache Hit!
| Returns (cookie="fbr_session=xyz", seller_profile={...})
v
[services/invoice_service.py: fill_invoice]
| 11. Checks item tax rates. If missing, calls adapter.fetch_sales_tax_rate(...)
| 12. Calls map_fbr_schema_to_internal_payload(input_data, seller_profile)
|     - Computes line math: value_sales_excluding_st, sales_tax_applicable, total_value
|     - Sets invoice status="draft" and total_amount
|
| 13. Calls DigitalInvoicingAdapter.create_or_update_invoice(cookie, internal_payload)
v
[adapter/client.py: create_or_update_invoice] ---> POST /api/invoices ---> [Digital Invoicing Software]
|
|<----- 201 Created {id: "remote_inv_987", status: "draft"} <----+
v
[services/invoice_service.py: fill_invoice]
| 14. Updates InvoiceJob(status="saved", remote_invoice_id="remote_inv_987")
| 15. Commits DB transaction & releases Redis distributed lock
v
[mcp_server/server.py] ---> Closes DB & Redis sessions
v
LLM Client receives:
{
  "status": "saved",
  "remote_invoice_id": "remote_inv_987",
  "summary": "Invoice draft created successfully with remote ID remote_inv_987."
}
```


5.3 Flow 3: Session Expiry, Cookie Caching, & 401 Reactive Refresh

```
[services/invoice_service.py] ---> calls adapter.create_or_update_invoice(stale_cookie, payload)
|
v
[Digital Invoicing Software]
|
| Returns 401 Unauthorized (session expired upstream)
v
[adapter/client.py] raises AuthenticationError("Session expired or unauthorized")
|
v
[services/invoice_service.py: fill_invoice] catches AuthenticationError!
|
| 1. Invalidate stale cache
v
[session/manager.py: invalidate_cookie]
| - Deletes Redis key: fbr_session:tenant:{tenant_id}
| - Deletes PostgreSQL row: tenant_sessions where tenant_id == tenant_id
|
v
[services/invoice_service.py: fill_invoice]
| 2. Re-requests valid cookie from SessionManager
v
[session/manager.py: get_valid_cookie_and_profile]
| - Redis & Postgres cache miss
| - Checks Redis login rate-limit flag (login_limit:tenant:{tenant_id})
| - Fetches encrypted credentials from tenant_credentials table
| - Decrypts email & password via security.decrypt_secret()
| - Calls adapter.login(email, password)
| - Receives fresh fbr_session cookie & Max-Age (e.g. 7199s)
| - Encrypts cookie & upserts tenant_sessions table
| - Stores cookie in Redis with TTL = (expires_at - now)
| - Returns (fresh_cookie, seller_profile)
v
[services/invoice_service.py: fill_invoice]
| 3. Retries adapter.create_or_update_invoice(fresh_cookie, payload)
v
[adapter/client.py] ---> POST /api/invoices ---> [Digital Invoicing Software]
|
|
|<----- 201 Created (Success!) <-----+
v
Invoice saved successfully without failing the caller turn!
```

End of Documentation Report.